

矩阵乘法优化

以前学习 CSAPP 的时候学过多线程。那时候都是用<pthread.h>库来实现，比较麻烦，需要手动管理线程的创建，参数传递还有同步，回收这些，比较麻烦。

我最开始就是先用 pthread 来写的，不过后来注意到了题目只能用 OpenMP，所以学习了一下 OpenMP 的用法。

发现 OpenMP 非常方便，直接把线程管理封装好了。只需要在线程创建的前面加上一句

```
#pragma omp parallel for
```

即可

那就非常简单了。这道题的思路也简单。我们要做的事情就是把 C 矩阵的每个元素算出来。C 中 i 行 j 列的元素在计算的时候需要用 A 的 i 行的所有元素，乘以 B 的 j 列的所有元素。那么自然而然想让一个线程来完成这件事情。所以每个线程传入 i, j 以及 A, B, C, m, n, k 这些必要的参数，然后计算就可以了。这是第一轮的代码

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

static char *slurp_stdin(void) {
    size_t cap = 1u << 22, len = 0;
    char *buf = (char *)malloc(cap);
    if (!buf) return NULL;
    for (;;) {
        if (len + 1 >= cap) {
            cap <<= 1;
            char *grown = (char *)realloc(buf, cap);
            if (!grown) { free(buf); return NULL; }
            buf = grown;
        }
        size_t got = fread(buf + len, 1, cap - len - 1, stdin);
        if (got == 0) break;
        len += got;
    }
    buf[len] = '\0';
    return buf;
}

static int next_long(const char **p, long *out) {
    char *end;
    long v = strtol(*p, &end, 10);
    if (end == *p) return 0;
    *p = end;
    *out = v;
    return 1;
}

static int next_float(const char **p, float *out) {
    char *end;
    float v = strtof(*p, &end);
    if (end == *p) return 0;
    *p = end;
    *out = v;
    return 1;
}

static float *read_matrix(const char **p, size_t count) {
    float *m = (float *)malloc(count * sizeof *m);
```

```

    if (!m) exit(1);
    for (size_t i = 0; i < count; ++i)
        if (!next_float(p, &m[i])) exit(1);
    return m;
}

static char obuf[1u << 20];

void calOne(float *a, float *b, double *c, int i, int j, int n, int k) {
    double sum = 0.0f;
    // 这个就是按照C，计算C的x行y列的元素
    // 也就是用A的i行，乘以B的j列
    for(long t = 0; t < n; ++t) {
        sum += (double)a[(size_t)i * (size_t)n + t] *
            (double)b[(size_t)t * (size_t)k + j];
    }

    c[(size_t)i * (size_t)k + j] = sum;
}

int main(void) {
    static char *input = NULL;
    input = slurp_stdin();
    if (!input) return 1;
    const char *p = input;

    long m, n, k;
    if (!next_long(&p, &m) || !next_long(&p, &n) || !next_long(&p, &k)) return 1;
    if (m < 1 || n < 1 || k < 1 || m > 4096 || n > 4096 || k > 4096) return 1;

    float *a = read_matrix(&p, (size_t)m * (size_t)n);
    float *b = read_matrix(&p, (size_t)n * (size_t)k);
    double *c = (double *)calloc((size_t)m * (size_t)k, sizeof *c);
    if (!a || !b || !c) return 1;

    // 把这里改成你的多线程实现

#pragma omp parallel for
    for (long i = 0; i < m; ++i) {
        for (long j = 0; j < k; ++j) {
            calOne(a, b, c, i, j, n, k);
        }
    }

    setvbuf(stdout, obuf, _IOFBUF, sizeof obuf);
    printf("%ld %ld\n", m, k);
    for (long i = 0; i < m; ++i) {
        const double *crow = c + (size_t)i * (size_t)k;
        for (long j = 0; j < k; ++j) {
            if (j) putchar(' ');
            printf("%.6f", crow[j]);
        }
        putchar('\n');
    }
    free(a);
    free(b);
    free(c);
    free(input);
    return 0;
}

```

顺带一提，几个 malloc 在我的 VSCode 会报错，所以我加了类型转换

进行评分

```
nsccl@login:~$ nsccl-run bash run_test.sh
execution 988e8726-9638-4b96-810a-7977812ed8a0
== AOI MATMUL CPU TIMED RUN ==
cwd=/workspace
identity=20035:20035
affinityCpus=16 requestedThreads=16
== build ==
gcc (conda-forge gcc 14.3.0-19) 14.3.0
build=ok
== statement sample check ==
2 2
58.000000 64.000000
139.000000 154.000000
sample=ok
== correctness check at 1024 scale (single vs multi byte-identical) ==
singleCksum=3954527176 11534346
multiCksum=3954527176 11534346
medium=ok
== generate timing input (2048 x 1024 x 2048) ==
-rw----- 1 aoiu_08ee3c755c2449b9b53a aoiu_08ee3c755c2449b9b53a 37748110 Oct 1
02:44 large.in
== single-thread baseline (OMP_NUM_THREADS=1) ==
singleMs=30695
== multi-thread run (OMP_NUM_THREADS=16) ==
multiMs=4452
== timing evidence ==
ratio=0.145 gate65=PASS
== AOI MATMUL CPU TIMED RUN END ==
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not permitted

aoi-execution-start 1790822634053011556
aoi-execution-end 1790822676780123175
aoi-execution-elapsed-ms 42727
execution 988e8726-9638-4b96-810a-7977812ed8a0: completed
```

可以看到时间从单线程的 30695 下降到了 4452,耗时为单线程的 14.5%，达标了

这是最朴素的方法，因为我之前学习 CSAPP 的时候记得一个局部性原理，也就是说，按列访问的那个矩阵，Cache Hit Rate 会很低，这一部分会拉低性能。所以我又想了一下之前学的另一种方法 $C[i][j] += A[i][t] * B[t][j]$

这方法的好处就是极大提高了 Cache Hit Rate，因为它可以对 A,B 都按行访问

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

static char *slurp_stdin(void) {
    size_t cap = 1u << 22, len = 0;
    char *buf = (char *)malloc(cap);
    if (!buf) return NULL;
    for (;;) {
        if (len + 1 >= cap) {
            cap <<= 1;
            char *grown = (char *)realloc(buf, cap);
            if (!grown) { free(buf); return NULL; }
            buf = grown;
        }
        size_t got = fread(buf + len, 1, cap - len - 1, stdin);
        if (got == 0) break;
        len += got;
    }
    buf[len] = '\0';
    return buf;
}
```

```

}

static int next_long(const char **p, long *out) {
    char *end;
    long v = strtol(*p, &end, 10);
    if (end == *p) return 0;
    *p = end;
    *out = v;
    return 1;
}

static int next_float(const char **p, float *out) {
    char *end;
    float v = strtof(*p, &end);
    if (end == *p) return 0;
    *p = end;
    *out = v;
    return 1;
}

static float *read_matrix(const char **p, size_t count) {
    float *m = (float *)malloc(count * sizeof *m);
    if (!m) exit(1);
    for (size_t i = 0; i < count; ++i)
        if (!next_float(p, &m[i])) exit(1);
    return m;
}

static char obuf[1u << 20];

void calOne(float *a, float *b, double *c, int i, int j, int n, int k) {
    // c[i][j] += a[i][t] * b[t][j]
    for(long t = 0; t < n; ++t) {
        c[(size_t)i * (size_t)k + j] += a[(size_t)i * (size_t)n + t] * b[(size_t)t *
(size_t)k + j];
    }
}

int main(void) {
    static char *input = NULL;
    input = slurp_stdin();
    if (!input) return 1;
    const char *p = input;

    long m, n, k;
    if (!next_long(&p, &m) || !next_long(&p, &n) || !next_long(&p, &k)) return 1;
    if (m < 1 || n < 1 || k < 1 || m > 4096 || n > 4096 || k > 4096) return 1;

    float *a = read_matrix(&p, (size_t)m * (size_t)n);
    float *b = read_matrix(&p, (size_t)n * (size_t)k);
    double *c = (double *)calloc((size_t)m * (size_t)k, sizeof *c);
    if (!a || !b || !c) return 1;

    // 把这里改成你的多线程实现

#pragma omp parallel for
    for (long i = 0; i < m; ++i) {
        for (long j = 0; j < k; ++j) {
            calOne(a, b, c, i, j, n, k);
        }
    }

    setvbuf(stdout, obuf, _IOFBF, sizeof obuf);
    printf("%ld %ld\n", m, k);
    for (long i = 0; i < m; ++i) {
        const double *crow = c + (size_t)i * (size_t)k;

```

```

        for (long j = 0; j < k; ++j) {
            if (j) putchar(' ');
            printf("%.6f", crow[j]);
        }
        putchar('\n');
    }
    free(a);
    free(b);
    free(c);
    free(input);
    return 0;
}

```

再测评一次

```

nsccl@loginin:~$ nsccl-run bash run_test.sh
execution fa9d64c7-7858-4181-87ae-db83052a8a58
== AOI MATMUL CPU TIMED RUN ==
cwd=/workspace
identity=20035:20035
affinityCpus=16 requestedThreads=16
== build ==
gcc (conda-forge gcc 14.3.0-19) 14.3.0
build=ok
== statement sample check ==
2 2
58.000000 64.000000
139.000000 154.000000
sample=ok
== correctness check at 1024 scale (single vs multi byte-identical) ==
singleCksum=594681800 11534346
multiCksum=594681800 11534346
medium=ok
== generate timing input (2048 x 1024 x 2048) ==
-rw----- 1 aoiu_08ee3c755c2449b9b53a aoiu_08ee3c755c2449b9b53a 37748110 Oct  1
03:15 large.in
== single-thread baseline (OMP_NUM_THREADS=1) ==
singleMs=33869
== multi-thread run (OMP_NUM_THREADS=16) ==
multiMs=4106
== timing evidence ==
ratio=0.121 gate65=PASS
== AOI MATMUL CPU TIMED RUN END ==
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not permitted

aoi-execution-start 1790824509047180561
aoi-execution-end 1790824554726522536
aoi-execution-elapsed-ms 45679
execution fa9d64c7-7858-4181-87ae-db83052a8a58: completed

```

只有略微提升，效果不是很明显，从 14.5% 优化到了 12.1%